

Exploring Sequential Modeling for Real-Time Multiclass Crime Detection

Kaniz Fatima Lima
Asian Institute of Technology
st126399@ait.asia

Supipi Karunathilaka
Asian Institute of Technology
st126489@ait.asia

Zwe Yu Ya Kyaw Zin Oo
Asian Institute of Technology
st125990@ait.asia

1. Abstract

This project aims to explore the effectiveness of sequential modelling which consists of spatial based models followed by temporal models, beyond single models for multiclass crime detection in real-time settings. Simply, it refers to developing an intelligent computer vision system that classifies the event in four categories, namely, Normal Situation, Shooting, Robbery, and Fighting. In the following sections, the crime detection system will be developed using various deep learning frameworks and methodologies.

2. Background

In recent years, crime detection has been a major challenge in many regions around the world, as countries have tried various methods to prevent crime and investigate crime scenes. One way to detect crime is through traditional surveillance, which requires humans to continuously monitor CCTV footage. Thus, to reduce human workload and improve efficiency, an automated process that provides a crime-detection system with high accuracy is necessary worldwide. In automation, a real-time crime detection system can identify suspicious or criminal activities, and it sends alerts to the authorities immediately when a crime is detected. Then this alert may be used by the responsible person to take the required action. However, as the risk differs with the type of crime, it is important to analyze the severity of the detected crime. To exemplify, shooting at somebody else or robbing a store with a knife is riskier and dangerous to people than a normal fight among them. Thus, by analyzing these fields thoroughly, it will give immense help to the authorities for advanced preparation in handling extreme crime cases.

3. Project Aim

Thus, this project aims to implement a real-time crime detection system that can detect and differentiate four common types of events, such as, normal (when no suspicious behavior or weapon is detected), shooting, robbery, fighting (without weapons)

4. Literature Review

4.1. Related Papers of weapon detection

The authors [1] and [2] focused on weapon detection by training models on images containing various types of weapons, however they cannot determine the actual occurrence of the crime.

4.2. The use of various real-time object detection models

Studies such as [3] show the effectiveness of lightweight models, where YOLO and **MobileNetV2** achieved 95% accuracy for real-time weapon and violence detection. Similarly, [4] implemented a real-time school and office monitoring system using webcams, microphones, and GPS, reporting high accuracy with minimal latency under diverse conditions. In [5], a hybrid ResNet50 + LSTM model reached 95.97% accuracy for classifying suspicious vs. non-suspicious activities. For multiclass crime analysis, [6] combined YOLOv5, Deep SORT, and LSTM to classify violent actions, achieving up to 96.1% accuracy across multiple benchmark datasets.

4.3. The use of deep learning models for the identification of crimes

Deep learning models have become widely used for crime detection due to their robustness and ability to learn complex patterns from raw data [7,8]. For multiclass classification, [9] showed that 3D-CNN and ConvLSTM significantly outperform standard CNNs, achieving validation accuracies of 0.991 and 0.821 for classifying no-crime, crime, and weapon-based crime.

4.4. Literature Gap

Hybrid spatial-temporal architectures, where CNNs extract spatial features and RNN/LSTM/GRU/ConvLSTM capture motion, have shown strong potential for crime detection [7]. However, most studies apply these methods only to binary classification, leaving a clear gap in multiclass hybrid crime detection and limited insights into real-time optimization on resource-constrained devices [5]. Recent works demonstrate strong multiclass performance us-

ing 3D CNNs [10], fusion models combining Conv-ViT, ResNet-50, and Conc-ViT with ISSOA optimization [11], and strategies such as dual-population mechanisms for better exploration and accuracy [12]. Yet, these methods remain insufficiently explored for scalable, real-time deployment, highlighting the need for efficient and practical multiclass crime-detection models.

5. Problem Statement

As this project mainly focuses on multiclass classification, the model should have the capability to identify spatial and behavioral patterns for each category and clearly classify any real-time video clip into which crime category. The selected categories (normal, fighting, robbery and shooting) have different moving patterns from each. For example, fighting moments are faster than shooting. Therefore, having a strong model for spatial and behavioral patterns will be considered in developing the project.

There are two common ways to do crime classification that are used:

- Clip-level classification (short windows → label fight/robbery/killing/none). This is good for trimmed clips or when the event can be assumed to be mostly inside the clip.
- Long untrimmed surveillance/anomaly detection (detect and temporally localize incidents in long streams). This is a more realistic approach for CCTV as it often uses weak supervision.

Between the two common approaches, the clip-level classification method is selected for this study as the models classify short video segments (e.g., 2–5 seconds) such as robbery, fighting, gun crime, or normal. This approach is more practical for real-time implementation because it allows the system to continuously process short video windows and produce immediate predictions.

In contrast, the long untrimmed or anomaly detection approach is less suitable for this project due to several limitations. It typically requires weakly labeled or unlabeled long surveillance videos, making model training complex and computationally expensive. Moreover, anomaly detection models often suffer from high false alarm rates since they rely on deviations from normal patterns rather than explicit class definitions. They are also harder to deploy in real time because of the need for temporal localization and large memory usage during continuous monitoring.

Therefore, the clip-level classification approach offers a more accurate, efficient, and feasible solution for real-time crime detection, while the long-untrimmed approach, though more realistic, is less practical for immediate system deployment within this project's scope.

6. Objectives

- As the approach is stated to have sturdy spatial and behavioral patterns along with accurate deliverables, the goal of this project is to explore the effectiveness of sequential modeling beyond single models for multiclass crime detection in real time.
- Another objective is to initiate the comparison of single-stage and two-stage models for the multiclass crime classification in terms of accuracy and in real time scenario.

7. Dataset

A total of 240 videos were collected from “UCF Crime,” belonging to four categories: normal (60) videos for different event categories, shooting (60), robbery(60), fighting (60). Videos ranged between 5–25 seconds and covered diverse lighting conditions, environments, and camera angles to improve model generalization. Low-quality or blurry videos were removed after a preliminary screening step. Each video was manually annotated according to its category, and labels were verified by multiple reviewers to ensure consistency. Only publicly accessible content was used, ensuring no violation of privacy or ethical guidelines. The dataset used here is a collection of large real surveillance videos with anomaly classes including fighting, robbery, shooting, etc., that is good for realistic long-video anomaly setups [13].

8. Methodology

In this study, the clip-level classification method is selected firstly, because Clip-level models classify short video segments (e.g., 2–5 seconds) such as robbery, fighting, gun crime, or normal. These focus on taking a short video clip (e.g., 16–64 frames) and assigning one label to the whole clip. Before training, videos are annotated by:

- saving each video's name to a cache
- creating a text file replica (annotation index)

For each video, OpenCV reads the clip. Frames are extracted and stored in a list. Then, use a “frame interval” (not every frame)

$$\text{frame interval} = \frac{\text{total_frames}}{\text{required_sequence_length}}$$

Extracted frames are resized to 224×224 or 64×64 depending on the model. Reason: smaller frames → faster training and less GPU memory. For CNN-based models, frames are used in two ways:

- Using Inception-V3 as feature extractor Baseline settings described:

- weights imagenet
- include_top false
- pooling avg input_shape = (frame_size, frame_size, 3)

This extracts features per-frame before classification.

- Or using raw frames directly CNN, 3D-CNN, ConvLSTM perform feature extraction inside the model.

8.0.1 Deep Learning Models

Three deep learning models (CNN, 3D-CNN, ConvLSTM) are chosen and trained to classify three types of videos: CV – Crime Video, NCV – Non-Crime Video, WCV – Weapon-Crime Video

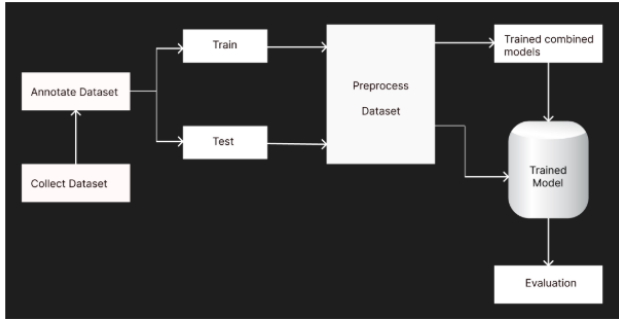


Figure 1. Flow of Proposed System

9. Models

9.1. Baseline

In this project, CNN, ConvLSTM and 3D CNN were used as baseline models tested for multiclass classification. They have shown better performance in multiclass classification with a small number of datasets in a past study. The other reason for taking them as baselines for our project is that these models act as one stage model for classification. But our approach is to investigate how two stage models affect multiclass classification with a rare dataset.

9.2. Resulted Models

- Resnet50 + LSTM
- Resnet50 + RNN
- Resnet50 + GRU
- MobileNetV2 + LSTM
- MobileNetV2 + RNN
- MobileNetV2 + GRU

10. Model Architectures Backbone+GRU

The study evaluates several end-to-end video classification architectures that combine spatial feature extraction with temporal modeling. Pretrained 2D CNN backbones (ResNet18, ResNet50, MobileNet-V2) generate frame-level features, which are then passed as sequences to either an LSTM (hidden size 128) or GRU (hidden size 256) to capture temporal dynamics in the CNN-LSTM and CNN-GRU models. Beyond these recurrent approaches, two fully spatiotemporal models were implemented: ConvLSTM, which preserves spatial structure while modeling temporal transitions through convolutional recurrent cells, and 3D-CNN, which applies volumetric convolutions to learn motion patterns directly from raw video clips. Together, these models enable a comparative analysis of hybrid 2D-temporal pipelines and fully 3D spatiotemporal learning.

10.1. Training Configuration

All models were trained under a unified pipeline with a 60/20/20 stratified split to preserve class balance. Videos were uniformly sampled, resized to 224×224, normalized with ImageNet statistics, and loaded in mini-batches of four using PyTorch. Training used the AdamW optimizer, cross-entropy loss with optional label smoothing, and early stopping to prevent overfitting; GRU models applied a three-epoch warm-up and cosine-annealing scheduler. Backbone strategies included full freeze, partial freeze, or delayed unfreezing, with mixed-precision and optional gradient checkpointing to reduce memory usage. Performance was measured using overall and per-class accuracy, along with confusion matrices.

10.2. Model Architecture Backbone+LSTM

After each frame is processed individually by the CNN, its output feature map is globally average-pooled to obtain a single feature vector. For a sequence of TTT frames, the output becomes a tensor of shape (B,T,D)(B, T, D)(B,T,D), where D is the CNN feature dimension (e.g., 2048 for ResNet50, 512 for ResNet18, and 1280 for MobileNet-V2). This sequence is then passed into an LSTM layer with a hidden size of 128 and one recurrent layer. The LSTM encodes temporal patterns and outputs a hidden representation at each timestep; only the final hidden state is used for classification. The classification head consists of two fully connected layers with ReLU activation and dropout regularization, followed by a final linear layer that maps to the four output classes (Normal, Fighting, Robbery, and Shooting) All Backbone-LSTM models were trained under a consistent and reproducible training pipeline. Video annotations were first loaded from a CSV file, and the dataset was split using a stratified 60–20–20 division into training, validation, and testing sets. Each video was processed using uniform frame sampling, ensuring that a fixed number

of frames ($T=16$, $T=16$, $T=16$ by default) were extracted regardless of video length. Frames were resized to 224×224 and normalized using ImageNet mean and standard deviation to match the backbone's training distribution. Training was performed using AdamW optimizer and weight decay, Cross-entropy loss with optional label smoothing was used as the optimization objective. A batch size of four was selected due to GPU memory constraints, and all experiments were run on either CPU or GPU depending on device availability. To further reduce memory usage, automatic mixed precision (AMP) training was enabled when specified. The training loop included both training and validation phases per epoch, with real-time progress tracked using tqdm and TensorBoard logging. Early stopping with a patience of six epochs was employed to prevent overfitting. The best-performing model (based on validation accuracy) was saved for each backbone-freeze configuration. During evaluation, the model was tested on unseen videos using the same frame extraction and preprocessing pipeline. Per-class accuracy and overall accuracy were computed, and confusion matrices were generated to analyze misclassification patterns. A pretrained backbone models for CNN; ResNet50 (string backbone of CNN) ResNet18, and MobileNetV2 (light weight) were explored to extract spatial features from each video frame. MobileNetV2 (fast, efficient, lightweight) that uses depthwise convolution layers for faster classification tasks and can be easily deployed, Depthwise separable convolutions significantly decrease computational requirements compared to conventional layers. The CNN output flattened feature vectors and it is then fed sequentially to the next stage.

10.3. Sequence Modeling (Temporal Domain)

The extracted frame features are input into an RNN-based network that are LSTM, GRU that learns the temporal dependencies between frames. This helps identify motion patterns such as “fighting,” “running,” “pulling out a gun,” etc. Both Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks are widely used for temporal feature detection because they can model how frame-level features evolve over time in a video sequence. LSTMs use three gating mechanisms and a separate cell state, enabling them to capture more complex and long-range temporal dependencies, which can be beneficial for recognizing detailed or extended action patterns. However, this additional complexity makes LSTMs heavier and slower to train. In contrast, GRUs use a simplified two-gate structure and combine the cell and hidden states, resulting in fewer parameters and faster computation while still maintaining strong temporal modeling capabilities.

10.4. Classification Layer

The final output from the recurrent unit is passed to fully connected layers and a softmax classifier to predict categories like, Robbery, Fighting, Gun Crime, or Normal.

11. Preprocessing

11.1. Frame Sampling

Each video clip is standardized by sampling a fixed sequence of 30 frames. If a video contains fewer than 30 frames, interpolation is applied to generate evenly spaced frame indices, ensuring full coverage of the clip. For longer videos, 30 frames are uniformly selected across the entire duration. This approach maintains consistent input dimensions for the model while preserving the temporal structure of the original video.

11.2. Temporal Jitter Augmentation

Temporal jittering is applied during training by randomly shifting each sampled frame index by ± 2 frames. This helps the model handle variations in action speed, frame rates, and natural motion differences, improving overall robustness.

11.3. Frame-Level Transformations

Each extracted frame undergoes standard preprocessing, including conversion from BGR to RGB, resizing to 224×224 pixels, and normalization using ImageNet mean and standard deviation. The frames are then converted into PyTorch tensors.

11.4. Classification Network

The output from the LSTM is passed through a series of fully connected layers for classification. It first goes through a dropout layer to reduce overfitting, followed by a $128 \rightarrow 128$ fully connected layer with a ReLU activation. Another dropout layer is applied before the final output layer, which maps the features to four class scores. A softmax function then converts these scores into probabilities corresponding to the four crime categories.

11.5. Training Strategy

To address class imbalance in the dataset—such as the lower number of Shooting samples—a Balanced Batch Sampler is used to ensure that each batch contains an equal number of samples from all classes, with minority classes oversampled through replacement. This reduces model bias and improves multiclass performance. Training uses a weighted Cross-Entropy Loss, where class weights are computed from the training split, and optional label smoothing helps reduce overconfidence. The model is optimized using AdamW with a learning rate of $1e-3$ and weight decay of $1e-4$, while mixed-precision training can be enabled

for faster computation. Early stopping is applied with a patience of eight epochs, halting training when validation accuracy no longer improves to prevent overfitting.

12. Results and Evaluation

Model	Test Accuracy
Resnet50 (freeze-full)	0.64
Resnet50 (freeze-partial)	0.6
ResNet18 (freeze-full)	0.56
ResNet18 (freeze-partial)	0.50
Mobilenet-v2 (freeze-full)	Overfitted from begining
Mobilenet-v2 (freeze-partial)	Overfitted from begining

Figure 2. Comparisns using GRU

All the models were overfitted. Except for Mobilenetv2,

Model	Test Accuracy
Resnet50 (freeze-full)	0.6 (overfit)
Resnet50 (freeze-partial)	0.64 (overfit)
ResNet18 (freeze-full)	0.58 (overfit)
ResNet18 (freeze-partial)	0.54 (overfit)
Mobilenet-v2 (freeze-full)	0.58
Mobilenet-v2 (freeze-partial)	0.54

Figure 3. Comparisns using LSTM

other models were overfitted. As each models were tested with freezing fulling and partially, only for Mobilenetv2,

in both configurations model performance was better than others. After the data augmentation , the improved best accuracy was 0.64 for MobileNetV2+LSTM with fully frozen layers and 0.60 for Mobilenetv2+LSTM with partially frozen layers. Even though baseline models were not

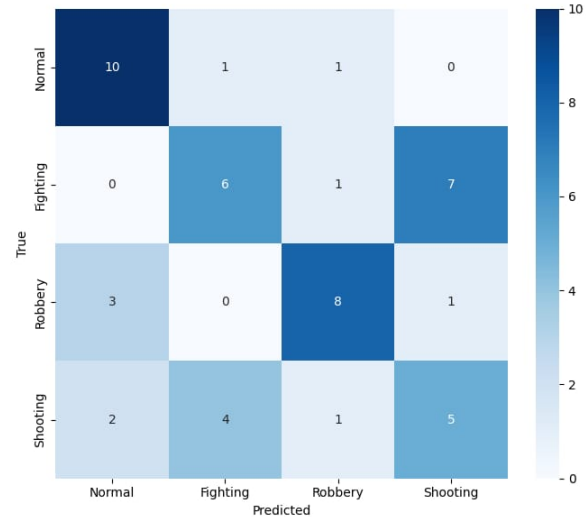


Figure 4. MobileNetV2 - Freeze Partial

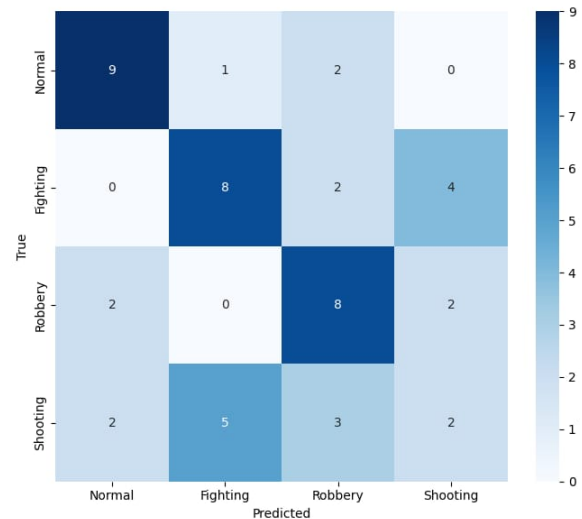


Figure 5. MobileNetV2 - Freeze full

overfitted, they showed accuracy only around 40

12.1. Visualizations

12.1.1 Baseline Models

12.1.2 Resnet50 and Resnet18 Using GRU

12.1.3 Resnet50 and Resnet18 Using LSTM

13. Discussion

The experiments show clear differences in how architectures handle video classification, particularly regarding

Model	Test Accuracy
Resnet50 (freeze-full)	0.6 (overfit)
Resnet50 (freeze-partial)	0.64 (overfit)
ResNet18 (freeze-full)	0.58 (overfit)
ResNet18 (freeze-partial)	0.54 (overfit)
MobileNet-v2 (freeze-full)	0.58
MobileNet-v2 (freeze-partial)	0.54

Figure 6. Accuracy with LSTM

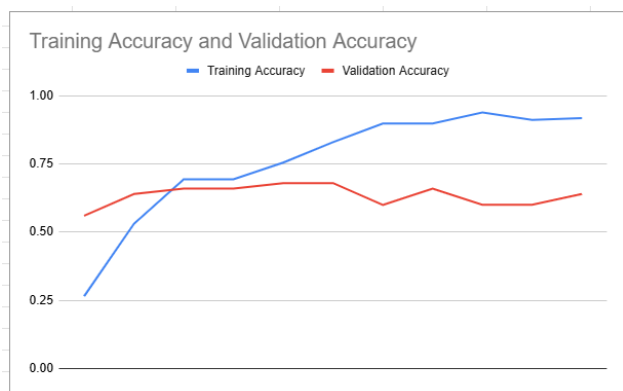


Figure 7. Comparison of Training accuracy and Validation accuracy using CNN Model

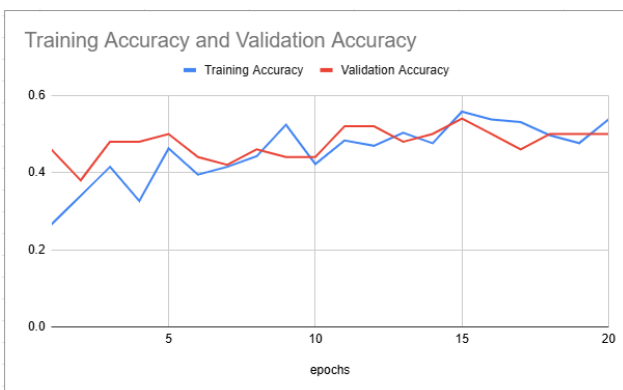


Figure 8. Comparison of Training accuracy and Validation accuracy using 3D-CNN Model

overfitting and generalization. Models with GRU suffered severe overfitting, achieving high training accuracy but poor validation performance, indicating difficulty in capturing

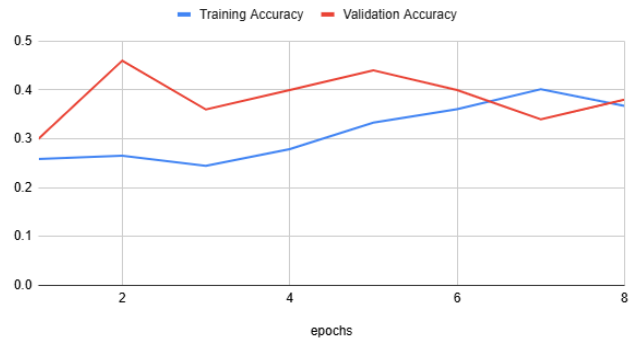


Figure 9. Comparison of Training accuracy and Validation accuracy using ConvLSTM Model

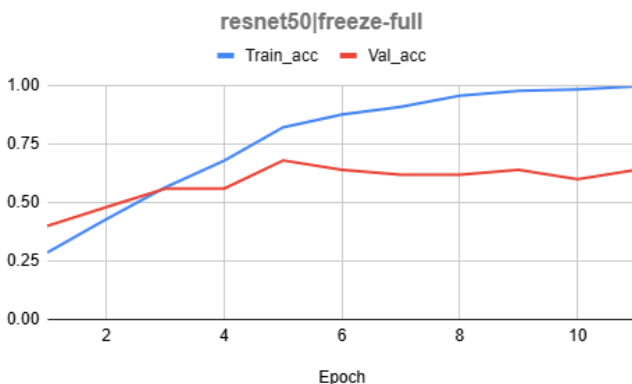


Figure 10. Comparison of Training accuracy and Validation accuracy GRU by Resnet50-full

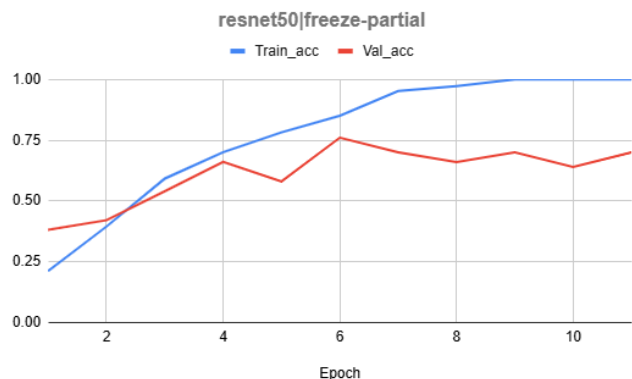


Figure 11. Comparison of Training accuracy and Validation accuracy GRU by Resnet50-Partial

complex temporal dynamics. LSTM-based models also overfitted, except for MobileNetV2+LSTM, which generalized better. With data augmentation, fully frozen MobileNetV2+LSTM reached 0.64 accuracy, outperforming other LSTM models, while partially frozen achieved 0.60. Confusion matrices reveal that MobileNetV2+LSTM reliably distinguishes Normal and Fighting classes but struggles with visually similar classes like Shooting and Robbery. The fully frozen variant achieved higher true-positive rates, suggesting that partial fine-tuning may introduce noise rather than useful features. Baseline models lack-



Figure 12. Comparison of Training accuracy and Validation accuracy GRU by Resnet18-full



Figure 13. Comparison of Training accuracy and Validation accuracy GRU by Resnet18-Partial

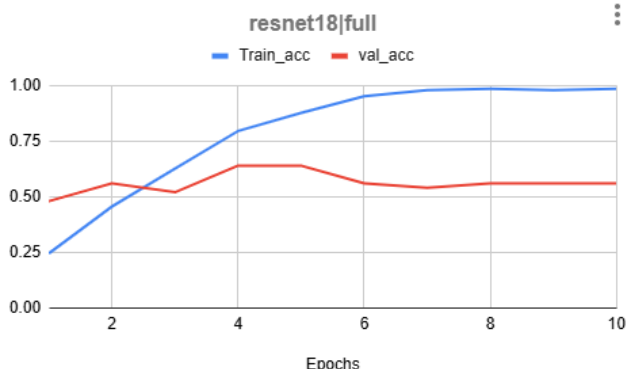


Figure 14. Comparison of Training accuracy and Validation accuracy GRU by Resnet18-full

ing temporal modeling achieved only 40% accuracy, highlighting the importance of temporal learning for violent activity detection. Overall, the results emphasize three insights: (1) temporal modeling is crucial, (2) lightweight backbones like MobileNetV2 generalize better, especially with LSTMs and augmentation, and (3) fully freezing the backbone can outperform partial freezing on small datasets. MobileNetV2+LSTM emerges as the most efficient and reliable configuration under dataset and resource constraints.

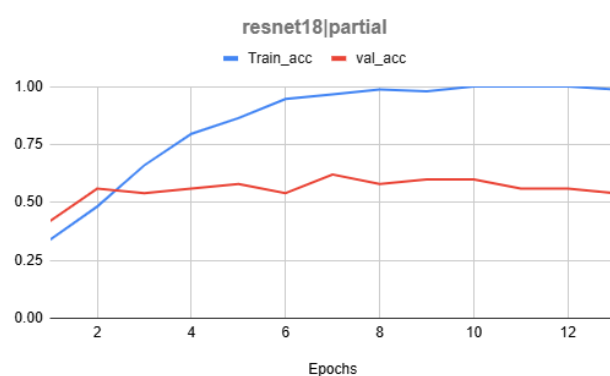


Figure 15. Results of the model Resnet18-Partial using LSTM

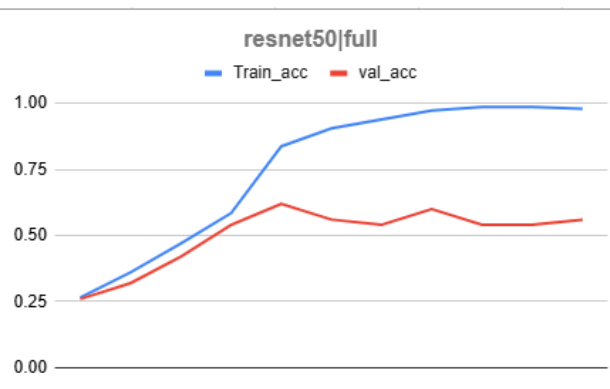


Figure 16. Results of the model Resnet18-full using LSTM

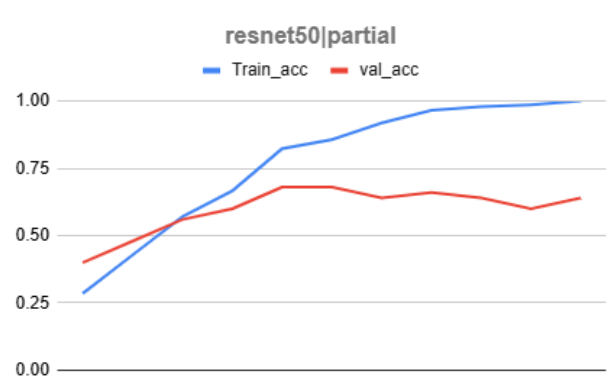


Figure 17. Results of the model Resnet18-Partial using LSTM

14. Limitations

Video Dataset findings is the key limitation

15. App Implementation

15.1. Visualization of Home Page

15.2. Visualization of Video Upload Page

The home page of the SheDetects Crime web application displays the app name, a project quote, and key features with icons for easy usability. A prominent “Start Detection” button directs users to the video upload page, where

videos can be dragged or uploaded for analysis. After clicking “Analyze Video”, results are generated within seconds: green with a tick indicates a normal video, while orange with an exclamation mark signals a crime-related class. The use of colors and icons ensures quick, intuitive recognition for all users.

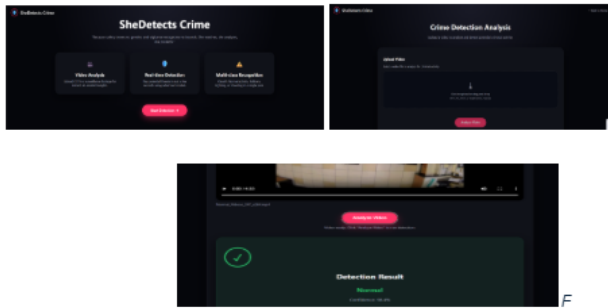


Figure 18. App Implementation

16. Conclusion

In this study, we addressed the challenge of multi-class classification on a small dataset. We compared traditional one-stage modeling with a two-stage modeling approach. The results indicate that two-stage modeling, where the problem is decomposed into sequential subtasks, can better handle class imbalance and extract more discriminative features than a single-stage model. Although limited by the small dataset size, the two-stage approach showed improved generalization and higher predictive performance across most classes. This suggests that for small datasets with multiple classes, breaking the problem into manageable stages can be a more effective strategy than relying solely on one-stage classification.

References

- [1] M. Boukabous and M. Azizi, “Image and video-based crime prediction using object detection and deep learning,” *Bulletin of Electrical Engineering and Informatics*, vol. 12, no. 3, pp. 1630–1638, 2023.
- [2] H. Jain, A. Vikram, A. Mohana, A. Kashyap, and A. Jain, “Weapon detection using artificial intelligence and deep learning for security applications,” in *2020 International Conference on Electronics and Sustainable Communication Systems (ICESC)*, pp. 193–198, IEEE, 2020.
- [3] M. M. Mukto *et al.*, “Design of a real-time crime monitoring system using deep learning techniques,” *Intelligent Systems with Applications*, vol. 21, p. 200311, 2024.
- [4] A. K. Pasi, L. Palarapu, A. Mailaram, L. P. Kanithi, and D. Bommana, “Real-time crime detection system using yolo,” in *Proceedings of the Conference on Advanced Computing and Intelligent Engineering*, pp. 570–579, 2025.
- [5] R. Shenoy, D. Yadav, H. Lakhotiya, and J. Sisodia, “An intelligent framework for crime prediction using behavioural tracking and motion analysis,” in *2022 International Conference on Emerging Smart Computing and Informatics (ESCI)*, pp. 1–6, IEEE, 2022.
- [6] N. Sivakumaran *et al.*, “An intelligent crime surveillance video system for real-time applications,” *Procedia Computer Science*, vol. 260, pp. 635–644, 2025.
- [7] V. Mandalapu, L. Elluri, P. Vyas, and N. Roy, “Crime prediction using machine learning and deep learning: A systematic review and future directions,” *IEEE Access*, vol. 11, pp. 60153–60170, 2023.
- [8] T. J. Nandhini and K. Thinakaran, “Detection of crime scene objects using deep learning techniques,” in *2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, pp. 357–361, IEEE, 2023.
- [9] R. C. Poonia, K. P. Upreti, P. P. P., and R. P. K., “Finding real-time crime detections during video surveillance by live cctv streaming using deep learning models,” in *Proceedings of the 2024 10th International Conference on Computing and Artificial Intelligence*, pp. 272–277, 2024.
- [10] A. Traoré and M. A. Akhloufi, “Violence detection in videos using deep recurrent and convolutional neural networks,” Sept.11 2024.
- [11] L. R. Vuyyuru and N. Purimelta, “Advancing automated street crime detection: A drone-based system integrating cnn models and enhanced feature selection techniques,” Aug.20 2024.
- [12] I. M. Nasir, M. Raza, J. H. Shah, M. A. Khan, Y.-C. Nam, and Y. Nam, “Improved shark smell optimization algorithm for human action recognition,” *Computers, Materials & Continua*, vol. 76, no. 3, pp. 2667–2684, 2023.
- [13] W. Sultani, C. Chen, and M. Shah, “Real-world anomaly detection in surveillance videos,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 6479–6488, 2018.